Java Class 2 — Variables, Data Types & User Input

After **installation + Hello World**, the natural next step is:

> **Variables → Data Types → Literals → Type Casting → Taking Input**

---

1. What is a Variable?

A **variable** is a named memory location used to store data.

For example:

```
int age = 25;
```

Here:

- `int` → data type
- `age` → variable name
- `25` → value
- `=` → assignment operator

Think of it as:

```
age
 ↓
┌──────────┐
│    25    │
└──────────┘
```

Basic syntax

```
dataType variableName = value;
```

Example:

```
int age = 25;
double salary = 50000.50;
char grade = 'A';
String name = "Rahul";
```

---

2. Why Do We Need Variables?

Without variables:

```
System.out.println("Rahul");
System.out.println(21);
System.out.println("Java");
```

With variables:

```
String name = "Rahul";
int age = 21;
String course = "Java";

System.out.println(name);
```

```
System.out.println(age);
System.out.println(course);
```

Variables allow us to **store and reuse information**.

---

3. Declaring a Variable

We can declare a variable without assigning a value.

```
int age;
```

This is called **declaration**.

Then:

```
age = 25;
```

This is **assignment**.

Together:

```
int age;
age = 25;
```

We can also do both at once:

```
int age = 25;
```

This is:

> Declaration + Initialization

---

4. Three Important Terms

Consider:

```
int age = 25;
```

Declaration

```
int age;
```

Initialization

```
age = 25;
```

Declaration + Initialization

```
int age = 25;
```

---

5. Java Data Types

Java data types are broadly divided into:

```
Data Types
│
├── Primitive
```

└── Non-Primitive / Reference

---

6. Primitive Data Types

Java has **8 primitive data types**:

Data Type	Example
byte	`byte x = 10;`
short	`short x = 100;`
int	`int x = 1000;`
long	`long x = 100000L;`
float	`float x = 10.5f;`
double	`double x = 10.5;`
char	`char x = 'A';`
boolean	`boolean x = true;`

These are extremely important for exams as well as programming.

---

7. Integer Data Types

There are four integer types:

```
byte
short
int
long
```

byte

```
byte age = 25;
```

Range:

```
-128 to 127
```

Size:

```
8 bits
```

---

short

```
short salary = 30000;
```

Size:

```
16 bits
```

Range:

```
-32,768 to 32,767
```

---

int

Most commonly used integer type.

```
int age = 25;
int marks = 95;
```

Size:

```
32 bits
```

Range approximately:

```
-2.147 billion to +2.147 billion
```

For normal whole numbers, use:

```
int
```

---

long

Used for very large integer values.

```
long population = 1400000000L;
```

Size:

```
64 bits
```

Notice the:

```
L
```

at the end.

Example:

```
long number = 10000000000L;
```

---

8. Floating-Point Data Types

Java provides:

```
float
double
```

float

```
float price = 99.50f;
```

Notice:

```
f
```

We normally write `f`/`F` for a float literal.

---

double

```
double salary = 55000.75;
```

`double` provides more precision than `float` and is commonly used for decimal values.

---

9. char

`char` stores a **single character**.

```
char grade = 'A';
```

Notice:

```
'A'
```

uses **single quotes**.

Correct:

```
char grade = 'A';
```

Incorrect:

```
char grade = "A";
```

Because `"A"` is a String.

---

10. String

`String` is used for text.

```
String name = "Akshay";
```

Notice the capital `S`:

```
String
```

and double quotes:

```
"Akshay"
```

Example:

```
String city = "Delhi";
String course = "Java Full Stack";
```

---

11. boolean

A boolean stores either:

```
true
false
```

Example:

```
boolean isStudent = true;
boolean isPassed = false;
```

Important:

```
boolean result = true;
```

Not:

```
boolean result = "true";
```

because `"true"` is a String.

---

12. Complete Primitive Data Type Table

Type	Size	Example
byte	8-bit	`byte a = 10;`
short	16-bit	`short a = 100;`
int	32-bit	`int a = 1000;`
long	64-bit	`long a = 1000L;`
float	32-bit	`float a = 10.5f;`
double	64-bit	`double a = 10.5;`
char	16-bit	`char a = 'A';`
boolean	JVM-dependent representation	`boolean a = true;`

Easy memory trick

```
Integer:
byte → short → int → long

Decimal:
float → double

Character:
char

True/False:
boolean
```

---

13. First Practical Program

Create:

```
Student.java
```

```
public class Student {

    public static void main(String[] args) {

        String name = "Rahul";
        int age = 21;
        double percentage = 87.5;
        char grade = 'A';
        boolean passed = true;

        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Percentage: " + percentage);
        System.out.println("Grade: " + grade);
        System.out.println("Passed: " + passed);
    }
}
```

Output:

```
Name: Rahul
Age: 21
Percentage: 87.5
```

```
Grade: A
Passed: true
```

---

14. What is String Concatenation?

Consider:

```
System.out.println("Age: " + age);
```

Suppose:

```
int age = 21;
```

Output:

```
Age: 21
```

The `+` joins the String and value.

This is called **String concatenation**.

---

15. Very Important + Concept

Look at:

```
System.out.println(10 + 20);
```

Output:

```
30
```

Because both are numbers.

But:

```
System.out.println("10" + 20);
```

Output:

```
1020
```

Because `"10"` is a String.

Another example:

```
System.out.println(10 + 20 + "Java");
```

Output:

```
30Java
```

But:

```
System.out.println("Java" + 10 + 20);
```

Output:

```
Java1020
```

Important rule

Once String concatenation starts from the left, subsequent + operations are generally treated as concatenation.

---

16. Taking Input from User

So far, we have written:

```
int age = 21;
```

But what if the user wants to enter their own age?

For this, we can use:

```
Scanner
```

---

17. Scanner

Java provides the `Scanner` class for reading input.

First import it:

```
import java.util.Scanner;
```

Then create a Scanner object:

```
Scanner sc = new Scanner(System.in);
```

Complete program:

```
import java.util.Scanner;

public class Student {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter your name: ");
        String name = sc.nextLine();

        System.out.print("Enter your age: ");
        int age = sc.nextInt();

        System.out.println("Name: " + name);
        System.out.println("Age: " + age);

        sc.close();
    }
}
```

---

18. Understanding Scanner

Import

```
import java.util.Scanner;
```

We are telling Java that we want to use the `Scanner` class.

Create object

```
Scanner sc = new Scanner(System.in);
```

Here:

```
Scanner → class
sc      → object/reference variable
System.in → keyboard input
```

Read String

```
String name = sc.nextLine();
```

Read integer

```
int age = sc.nextInt();
```

---

19. Scanner Methods

Some important methods:

Method	Input
`nextInt()`	Integer
`nextLong()`	Long
`nextDouble()`	Double
`nextFloat()`	Float
`next()`	One token/word
`nextLine()`	Complete line
`nextBoolean()`	Boolean

Examples:

```
int age = sc.nextInt();
```

```
double salary = sc.nextDouble();
```

```
String name = sc.next();
```

```
String address = sc.nextLine();
```

```
boolean result = sc.nextBoolean();
```

---

20. `next()` vs `nextLine()`

This is very important.

Suppose user enters:

```
Rahul Sharma
```

`next()`

```
String name = sc.next();
```

reads:

```
Rahul
```

nextLine()

```
String name = sc.nextLine();
```

reads:

```
Rahul Sharma
```

So:

```
next()     → one token
nextLine() → complete line
```

---

21. Example: Student Input

```
import java.util.Scanner;

public class Student {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter your name: ");
        String name = sc.nextLine();

        System.out.print("Enter your age: ");
        int age = sc.nextInt();

        System.out.print("Enter your percentage: ");
        double percentage = sc.nextDouble();

        System.out.println("\n--- Student Details ---");

        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Percentage: " + percentage);

        sc.close();
    }
}
```

---

22. Type Casting

Sometimes we need to convert one data type into another.

Example:

```
int a = 10;
double b = a;
```

This is allowed automatically.

This is called:

Widening conversion

```
int
 ↓
double
```

Narrowing Conversion

Suppose:

```
double a = 10.5;
int b = (int) a;
```

Output:

```
10
```

We explicitly tell Java:

```
(int)
```

This is called **explicit casting** or **narrowing conversion**.

---

23. Widening vs Narrowing

Widening

Usually automatic:

```
int x = 10;
double y = x;
```

```
int → double
```

Narrowing

Usually requires explicit casting:

```
double x = 10.5;
int y = (int) x;
```

```
double → int
```

---

24. Important Beginner Exercise

Ask students to create:

```
Employee.java
```

Take input for:

- Employee name
- Employee ID
- Age
- Salary
- Department

Then display:

```
========== EMPLOYEE ==========
Name:
```

```
ID:
Age:
Salary:
Department:
==============================
```

---

25. Class 2 Practice Questions

Basic

1. What is a variable?
2. What is the difference between declaration and initialization?
3. Name all 8 primitive data types.
4. What is the difference between `float` and `double`?
5. What is the difference between `char` and `String`?
6. What values can a boolean store?
7. What is type casting?
8. What is widening conversion?
9. What is narrowing conversion?
10. What is `Scanner`?

Predict the Output

Question 1

```
int a = 10;
int b = 20;

System.out.println(a + b);
```

Answer:

```
30
```

Question 2

```
System.out.println("10" + 20);
```

Answer:

```
1020
```

Question 3

```
System.out.println(10 + 20 + "Java");
```

Answer:

```
30Java
```

Question 4

```
System.out.println("Java" + 10 + 20);
```

Answer:

```
Java1020
```

Question 5

```
double x = 10.5;
int y = (int) x;

System.out.println(y);
```

Answer:

```
10
```

---

26. What Students Should Know After Class 2

By the end of this class, students should be able to:

✓ Create a Java file
✓ Create variables
✓ Understand primitive data types
✓ Use String
✓ Print variables
✓ Perform basic calculations
✓ Understand +
✓ Take input using Scanner
✓ Use next(), nextLine(), nextInt()
✓ Understand basic type casting
✓ Compile and run programs